Introducción

Contents

- Antecedentes de Spark: programación Distribuida y MapReduce
- Breve historia de Spark
- Apéndice: el formato de fichero Parquet

Antes de empezar con la tecnología de Apache Spark, que es el tema principal del curso, es necesario hacer un comentario sobre cómo nacieron las tecnologías Big Data. Repasaremos el problema del procesamiento masivo, que dio origen a Spark.

Antecedentes de Spark: programación Distribuida y MapReduce

Una vez que Google tenía resuelto el problema del almacenamiento de ficheros masivos en el sistema de ficheros distribuido Google File System (GFS), en 2004 publicaron un nuevo artículo sobre cómo aprovechar los datanodes para procesar estos ficheros que estaban almacenados de forma distribuida particionados en bloques. El artículo describe un paradigma de programación llamado MapReduce, que consiste en una manera abstracta de abordar los problemas para que puedan ser implementados y ejecutados sobre un cluster de ordenadores. Junto con el artículo, también liberaron una biblioteca de programación en lenguaje Java que implementaba este paradigma. Podemos definir MapReduce como

> Modelo abstracto de programación general, e implementación del modelo como bibliotecas de programación para procesamiento paralelo y distribuido de grandes datasets, inspirado en la técnica divide y vencerás.

Al mismo tiempo, en el año 2002, el ingeniero Doug Cutting desarrolló una conocida herramienta de búsqueda llamada Nutch. Cuando fue contratado por Yahoo en 2006, Doug Cutting lideró el proyecto Hadoop, que se independizó de Nutch e incorporó las ideas de los artículos de Google, creando así un ecosistema de herramientas y bibliotecas de programación para ejecutar MapReduce en clusters de ordenadores (principalmente en lenguaje Java). Yahoo donó Hadoop a la Apache Software Foundation en 2008 para

convertirlo en un proyecto open-source. El logotipo de Hadoop, un elefante amarillo, hace referencia a un elefante de peluche llamado Hadoop de una de las hijas de Doug Cutting.

La gran ventaja que proporciona tanto el modelo MapReduce como la implementación en forma de biblioteca, es que abstrae al programador de todos los detalles relativos al hardware, redes y comunicación entre los nodos, para que pueda centrarse solamente en el desarrollo de software para solucionar su problema.

El punto de partida son archivos muy grandes almacenados (por bloques) en HDFS (en aquel momento aún era GFS). ¿Podríamos aprovechar las CPUs de los datanodes del cluster para procesar en paralelo (simultáneamente) los bloques de un archivo? No queremos mover datos (el tráfico de red es muy lento): llevamos el cómputo (el código fuente de nuestro programa) donde están almacenados los datos (es decir, a los datanodes, en lugar de al revés, como ocurría en el modelo tradicional de aplicación). Las CPUs de los datanodes van a procesar (preferentemente) los datos que hay en ese datanode.

En el paradigma MapReduce, el usuario sólo necesita escribir dos funciones (enfoque divide y vencerás):

- Mapper: función escrita por el usuario e invocada por el framework en paralelo (simultáneamente) sobre cada bloque de datos de entrada (que generalmente coinciden con un bloque de HDFS), generando así resultados intermedios que están siempre en forma de (clave, valor) y son escritos también al disco local.
- Reducer: función del usuario invocada por el framework en paralelo para cada clave distinta, pasándole todos los valores que comparten esa clave.

Inconvenientes de MapReduce

MapReduce permite procesar los datos almacenados de manera distribuida en el sistema de archivos (por ejemplo HDFS) de un cluster de ordenadores. Permite resolver todo tipo de problemas, pese a que no siempre es sencillo pensar la solución en términos de operaciones map y reduce encadenadas. Por ello se dice que es de propósito general, a diferencia de, por ejemplo, el lenguaje SQL que está orientado específicamente a realizar consultas sobre los datos. Implementar en SQL algoritmos tales como un método de ordenación, o el algoritmo de Dijkstra para encontrar caminos mínimos en un grafo no sería posible. Sin embargo, MapReduce presenta varios inconvenientes, algunos muy serios:

- El resultado de la fase map (tuplas) se escribe en el disco duro de cada nodo, como resultado intermedio. Los accesos a un disco duro son aproximadamente un orden de magnitud (es decir, diez veces) más lentos que los accesos a la memoria principal (RAM)

de cada nodo, por lo que estamos penalizando el rendimiento debido a cómo está estructurado el propio framework.

- Después de la fase map, hay tráfico de red obligatoriamente (movimiento de datos, conocido como shuffle). De hecho, el movimiento de datos forma parte como una etapa obligada en el framework de MapReduce. En cualquier aplicación distribuida, el movimiento de datos de un nodo a otro constituye un cuello de botella y es una operación que debe evitarse si es posible.
- Por otro lado y relacionado con el punto anterior, para mover datos se necesita, en primer lugar, escribirlos temporalmente en el disco duro de la máquina origen, enviarlos por la red y escribirlos temporalmente en el disco duro de la máquina destino (el shuffle siempre va de disco duro a disco duro) para, finalmente, pasarlos a la memoria principal de dicho nodo.
- Estos dos inconvenientes se acentúan especialmente cuando el algoritmo que queremos implementar sobre un cluster es de tipo iterativo, es decir, requiere varias pasadas sobre los mismos datos para ir convergiendo. Este tipo de procesamiento es típico en algoritmos de Machine Learning, que es uno de los que más se puede beneficiar del procesamiento de grandes conjuntos de datos para extraer conocimiento. Se comprobó que sus implementaciones con MapReduce no eran eficientes debido a la propia concepción del framework.
- Finalmente, enfocar cualquier problema en términos de operaciones map y reduce encadenadas no siempre es fácil ni intuitivo para un desarrollador, y la solución resultante puede ser difícil de mantener si no está bien documentada.

En el año 2009 surgió en el grupo de investigación AMPLab, en la Univ. California en Berkeley, EEUU, una nueva propuesta para paliar estas deficiencias, que poco a poco ha ido imponiéndose hasta reemplazar completamente a Hadoop (más concretamente, a MapReduce).

> **Note**
>
> Apache Spark es un motor unificado de cálculo en memoria y un conjunto de bibliotecas para procesamiento paralelo y distribuido de datos en clusters de ordenadores

Analizando la definición:

- Es un *motor* de cálculo, esto es, un framework de propósito general orientado a resolver cualquier tipo de problema y con el que se puede implementar cualquier algoritmo.
- *Unificado*: el motor de cálculo es único e independiente de cómo utilicemos Spark:

    - Desde la API de DataFrames (para los lenguajes R, python, Java y Scala)
    - Desde herramientas externas que lanzan consultas SQL contra Spark (Hive, herramientas de Business Intelligence conectadas a Spark como Tableau, PowerBI, etc)
    - Desde una instrucción de la API de programación que recibe una consulta SQL como string, tan compleja como sea necesario
    - ...todo se traduce a un grafo de tareas al que Spark aplica optimizaciones de código automáticamente, y por tanto todos (API, aplicaciones BI, SQL, etc) se benefician de ellas
- *En memoria*: todos los cálculos se llevan a cabo en memoria, y solo se escriben resultados a disco (parciales o finales) cuando el usuario lo indica explícitamente (operación de guardado) o cuando la operación indicada por el usuario requiere forzosamente realizar movimiento de datos entre nodos (*shuffle,* el cual se lleva a cabo del disco duro del emisor al disco duro del nodo receptor). Además, el movimiento de datos solo se produce cuando es irremediable, y no de manera obligatoria. Esto consigue un rendimiento hasta 100 veces superior que MapReduce en tareas iterativas (varias pasadas sobre los mismos datos, como tareas de machine learning). La medición de tiempos en una regresión logística arrojó un factor de mejora de x100.

Breve historia de Spark

La Fig. 1 muestra un gráfico con la evolución temporal de Spark. Fue concebido como parte de la tesis doctoral de Matei Zaharia, un investigador en el grupo AMPLab de UC Berkeley. Tras la publicación en 2010 del primer artículo sobre Spark, y la liberación desde el principio del código fuente, se vio su potencia y la ganancia que se podría llegar a conseguir sustituyendo el procesamiento anterior en MapReduce por el procesamiento con este nuevo framework.

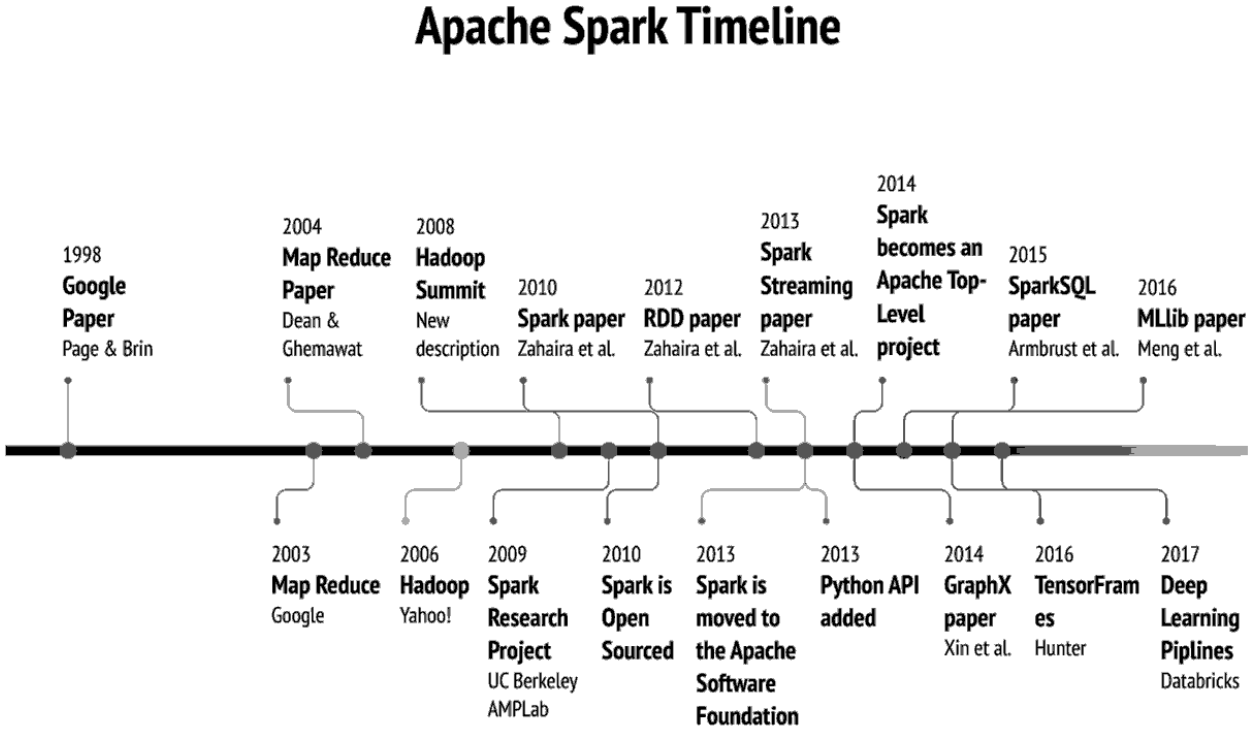


Fig. 1. Evolución de Apache Spark a lo largo del tiempo

Poco después del nacimiento de Spark, su creador Matehi Zaharia fundó la empresa Databricks, dedicada a dar servicios en torno a Spark, tales como como su propia plataforma Big Data y servicios cloud. Poco a poco se fue completando Spark con módulos específicos que aprovechaban esta tecnología para ofrecer API de mayor nivel de abstracción para tareas concretas, como procesamiento de datos en *streaming*, o la utilización de estructuras de datos distribuidas de más alto nivel y más similares a las tablas de una base de datos relacional, además de un módulo específico que implementa modelos de Machine Learning que pueden entrenarse con datos distribuidos.

Apéndice: el formato de fichero Parquet

Esta sección no está relacionada con Spark como tal, pero explica un formato de fichero muy frecuente para almacenar grandes volúmenes de datos, y que se utiliza muchísimo como fuente para leer datos masivos que luego se procesan utilizando algún framework distribuido, como por ejemplo Spark.

Apache Parquet es un formato de archivo nacido en 2013, de tipo columnar y comprimido (binario). Es el formato más utilizado para almacenar datos estructurados en HDFS (tablas o similar, incluyendo tipos complejos y anidados en las columnas). Es muy frecuente su uso en

conjunción con Apache Hive (tema siguiente). Parquet utiliza la técnica conocida como record-shredding para almacenar tipos de datos complejos y anidados (que un CSV no es capaz de representar). Sus principales características son:

- Almacena el esquema junto a los datos. También máx/mín por columna (resúmenes)
- Los valores de cada columna se guardan físicamente juntos (almacenamiento columnar). Beneficios:
  - Compresión de datos por cada columna para ahorrar espacio
  - Se pueden aplicar técnicas de compresión diferentes, específicas según el tipo de dato que almacena cada columna
  - Las consultas con filtrados sobre columnas concretas no necesitan leer toda la fila, mejorando el rendimiento
- Preparado para añadir más esquemas de codificación que se desarrollen en el futuro
- No requiere especificar separador de columnas, ni si se incluyen o no nombres de columna (siempre se incluyen por defecto)

Es con diferencia el formato más ampliamente utilizado para procesamiento en batch, tanto para datos de entrada como para guardar resultados, con datos estructurados, ya que puede ocupar entre 7 y 10 veces menos que un fichero CSV que almacene la misma información.